

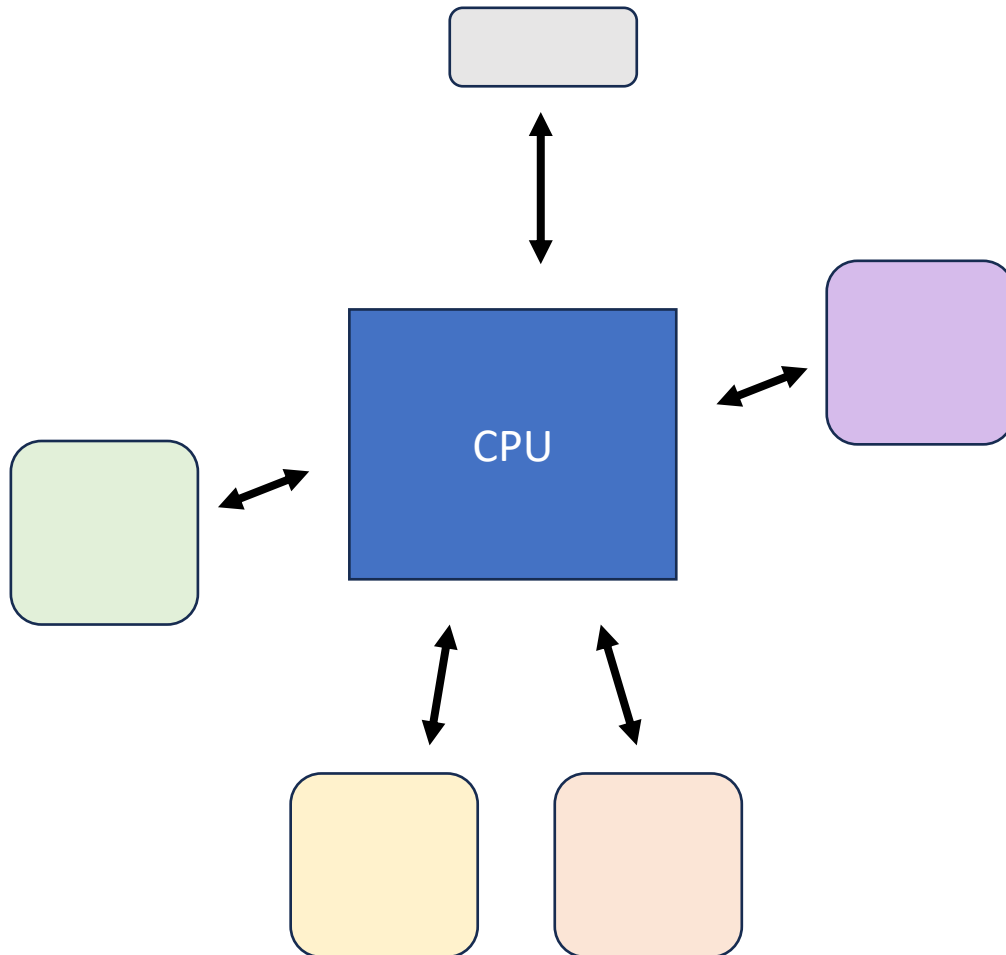


Devices

ECEN 427
Jeff Goeders

BYU Electrical & Computer
Engineering
IRA A. FULTON COLLEGE OF ENGINEERING

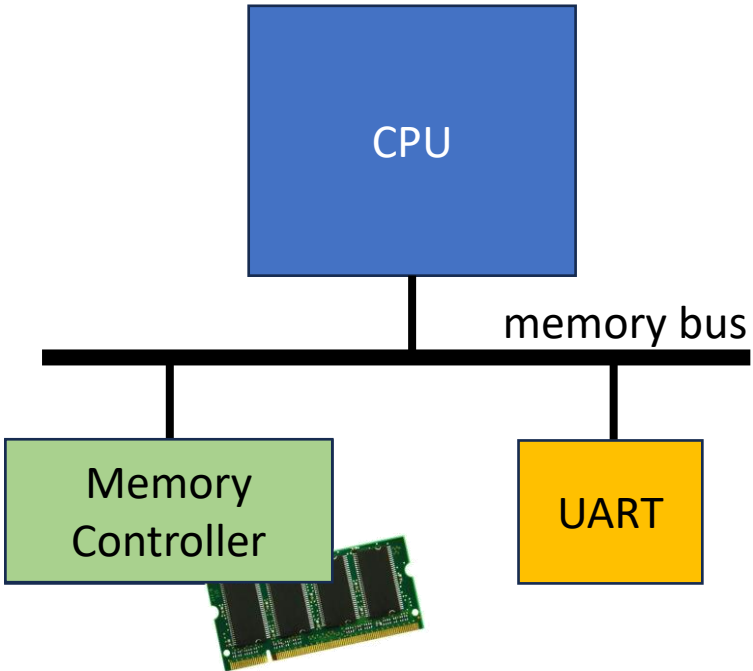
How does a CPU talk to other stuff?



- You learned about various CPU instructions in EC EN 224
 - Arithmetic (add/sub/mul)
 - Control (call/ret/jmp)
 - Data Movement (mov, push, pop)
- In x86 **mov** moves data between:
 - Register → Register
 - Immediate → Register / Memory
 - **Memory → Register Load / Read**
 - **Register → Memory Store / Write**
- Data movement to/from memory addresses is how a CPU talks to devices

Many other instruction sets (e.g. ARM) have dedicated operations for load/store

Load / Store Memory Operations

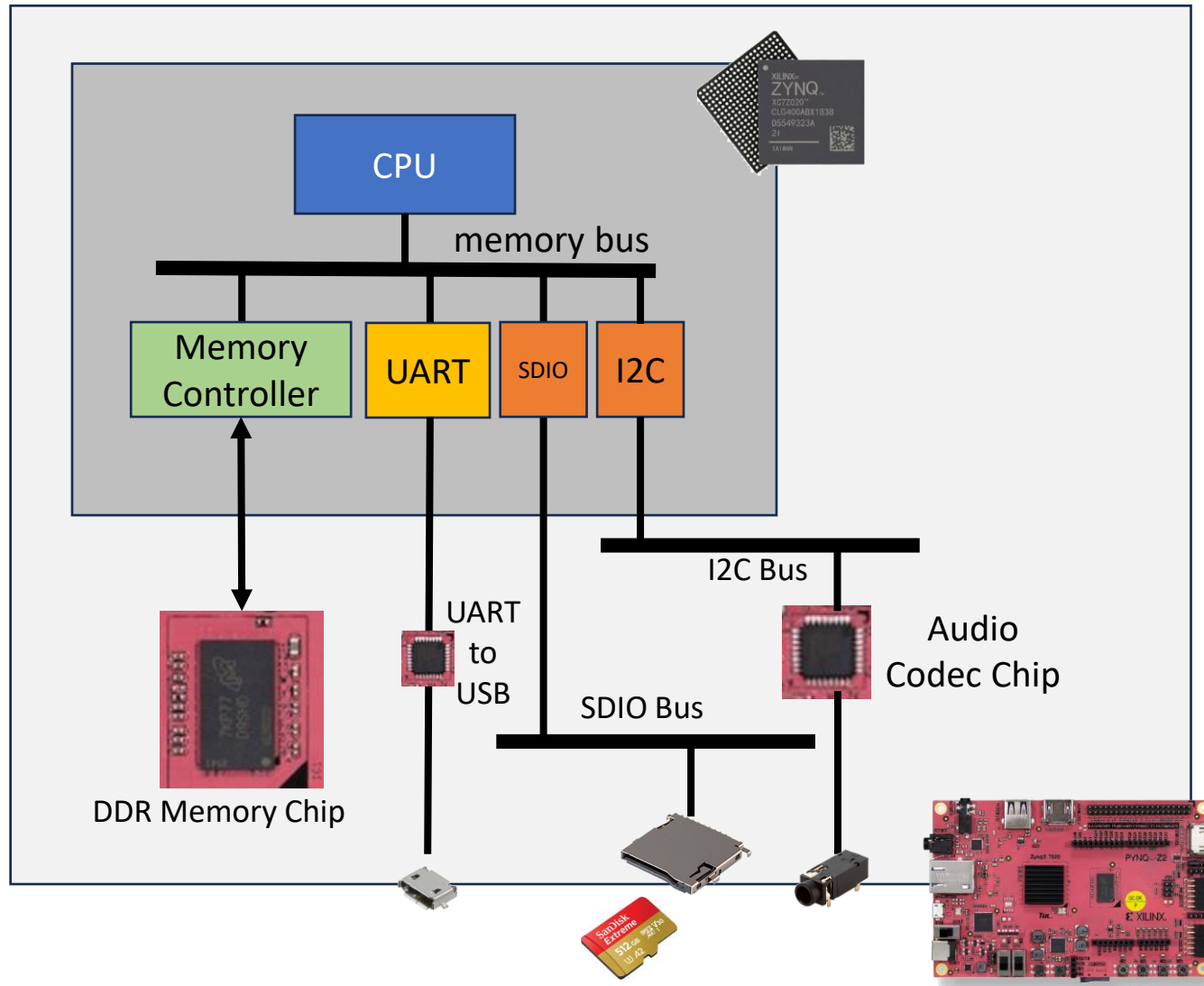


- **load** (read) and **store** (write) CPU operations trigger a **bus transaction**
 - Why? The CPU hardware is built to do this
 - We will learn more about buses later in the course, and you will design your own hardware on the bus.
- What C operations trigger a **load** and **store**?
 - Accessing variables? (sometimes; compiler may map our variables to registers or memory)

```
int x = 0;
```
 - Dereferencing pointers? (*usually* – see later slide)

```
int * x = <addr>;  
*x = 0;
```
- *Memory operations are not just to access memory; the CPU uses them to communicate with all devices on the bus*
- In fact, even when accessing memory, we are really communicating with the **memory controller**, which in turn interfaces with the actual memory device.

Context: On-Chip / Off-Chip Buses



The memory bus is contained **on-chip**;
designed for **intra-chip** communication.
(for most modern devices)

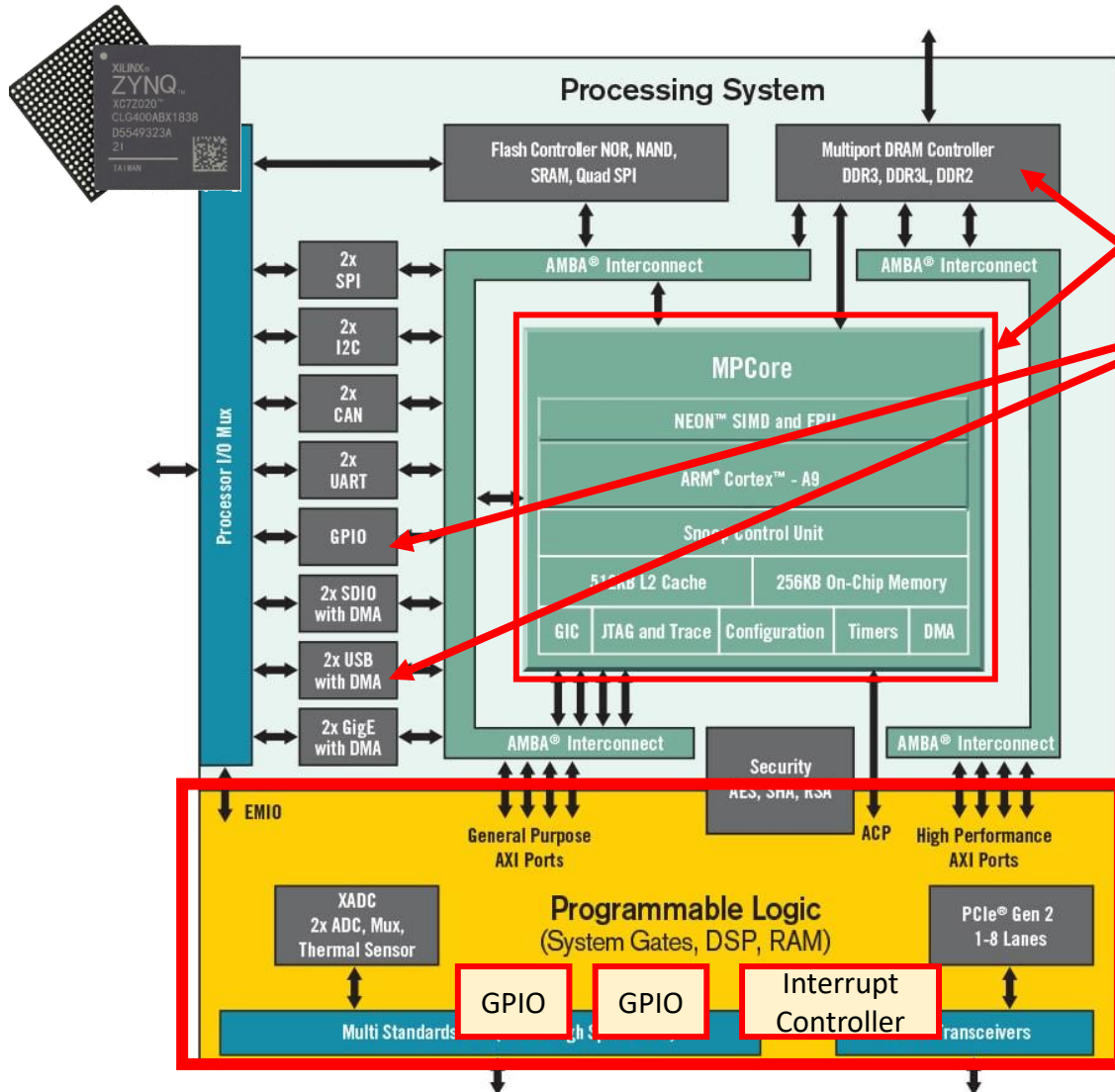
There are other bus protocols designed for
inter-chip communication (e.g. I2C, SPI)

- Longer distances; between chips on a printed circuit board (PCB)
- Typically slower than the memory bus

Technically, we have both an *internal* and *external* memory bus.

- We are primarily discussing the internal bus.
- External bus is the CPU device to DDR memory wiring

What we use in this class: Zynq-7000

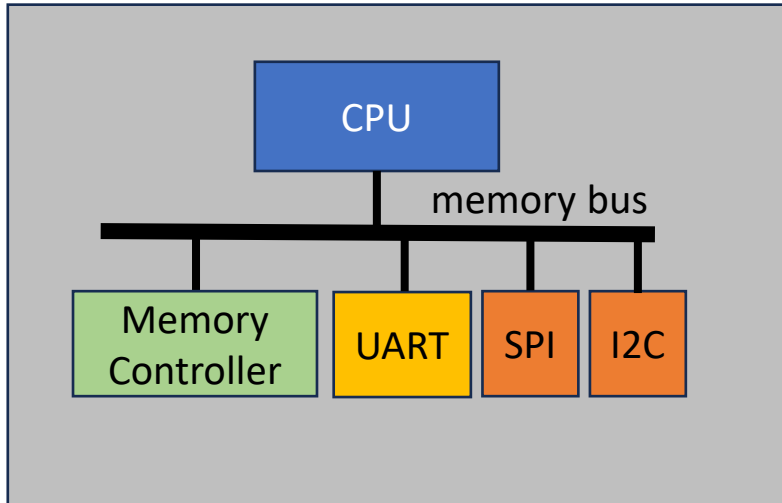


Zynq-7000 chip on the PYNQ Z-2 board

- The dual-core CPU
- The CPU has several “hard” blocks it interfaces with
 - We call these “IP” (Intellectual Property) blocks
- Various buses
- FPGA fabric
 - There are more “IP blocks” implemented in the FPGA bitstream given to you
 - Your system programs the provided bitstream as part of bootup
- What’s in the provided bitstream?

<https://byu-cpe.github.io/ecen427/documentation/hardware/>

The Address Map



Address space refers to the full set of addresses available to the CPU.

- Zynq-7000 has ARM Coretex-A9 CPU that uses 32-bit addresses, so a $2^{32} = 4\text{GB}$ address space.

The **address map** lists how the address space is split up and assigned to devices on the memory bus.

Loads/stores will be routed to the appropriate device based on this mapping.

Address	Size	Device
0x0000_0000	512 MB	DDR memory
0x4121_0000	64 KB	RGB LEDs GPIO
0x4122_0000	64 KB	Buttons GPIO
0x4123_0000	64 KB	Switches GPIO
0x4124_0000	64 KB	LEDs GPIO
0x4180_0000	64 KB	Interrupt controller
...	...	...

Important notes:

- **System memory is part of this address space** (DDR for our board). In some embedded systems you might see multiple memories.
 - E.g., microcontroller that uses addresses in flash for the program, and addresses in SRAM for data.
- **These are physical addresses.** We can assign the values to pointers to access them in a bare metal system, but not in a system with virtual memory.
- **The address space is not full.** There are many “gaps”. If you try to perform a load/store on an address with no device at the other end, results may range from exception to full system hang.

Inside a Device

- Inside a device block, individual addresses are **device registers**.
- In the hardware we use, all registers are 32-bits wide.
 - Means we perform a 32-bit load/store
 - Doesn't mean all bits in the register are meaningful
- The register space for a device may have many gaps:
 - Example: 64KB assigned, hardware only has a few 32-bit registers
 - If you load/store other addresses, behavior is IP-dependent
 - Throw an exception
 - Only use lower bits of an address and access a different memory location
 - Not respond and hang

Address	Size	Device
0x0000_0000	512 MB	DDR memory
0x4121_0000	64 KB	RGB LEDs GPIO
0x4122_0000	64 KB	Buttons GPIO
0x4123_0000	64 KB	Switches GPIO
0x4124_0000	64 KB	LEDs GPIO
0x4180_0000	64 KB	Interrupt controller
...	...	...

Offset	Register	Meaning
0x000	GPIO_DATA	Pin values
0x004	GPIO_TRI	Direction (not present on our blocks)
0x11C	GIER	Global interrupt enable
0x120	IP_ISR	Interrupt status
0x128	IP_IER	Interrupt enable

```
int * p = 0x41220128; // 0x4122_0000 + 0x128

*p = 1;                // Store 1 to register
```

**When you write to memory,
you're not writing to memory.**

*You're sending a message to a device on a bus.
What happens next is up to the device.*

The Illusion of Memory

In C, this looks like writing to RAM:

```
*ptr = 0x1;
```

But the CPU doesn't know what's there.

- It puts the address and value on the bus and moves on.
- The device on the other end decides what to do with it.

Devices Can Do Anything. A write might:

- Store a value (RAM — the boring case)
- Trigger a hardware reset
- Start a DMA transfer
- Toggle a GPIO pin
- Clear an interrupt
- Be silently ignored

The load/store is a message to the device

```
*ptr = 0xFF;  
int x = *ptr; // x might not be 0xFF
```

A device is free to return whatever it wants on a read:

- A status register returns current hardware state
- A FIFO register returns the next item in a queue
- A counter register returns a value that changes every read
- A write-only register returns 0 (or garbage)

Memory-mapped I/O is a conversation, not a variable.

```
*ptr = 0xFF;  
int y = *ptr; // compiler: "I just wrote 0xFF, y must be 0xFF"  
              // → optimizes away the read entirely
```

The Volatile Keyword

The compiler is allowed to optimize away memory accesses if it can prove the value is already known:

```
int x = *ptr;  
int y = *ptr; // compiler may reuse the loaded value in a register  
              // – only one actual load instruction generated
```

Or if the result is never used:

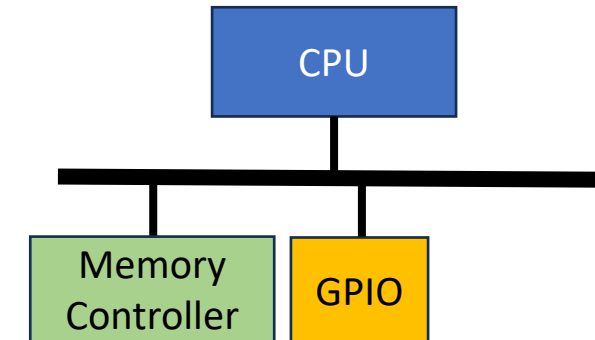
```
*ptr = 5;  
*ptr = 10; // first store may be eliminated entirely
```

volatile tells the compiler: "don't optimize this away, always really access memory":

```
volatile int *ptr = ...;  
int x = *ptr;           // guaranteed load every time  
*ptr = 5;               // guaranteed store every time
```

In Summary

- The CPU communicates with devices through **load** and **store** operations to the device registers
- Each device register is at a unique address across the entire address space. System memory is also part of this address space.
- The **load** and **store** operations should be thought of as *messages* to/from the device.
- The device may behave like a memory, or it may have more complex behavior.



Address	Size	Device
0x0000_0000	512 MB	DDR memory
0x4121_0000	64 KB	RGB LEDs GPIO
0x4122_0000	64 KB	Buttons GPIO
0x4123_0000	64 KB	Switches GPIO
0x4124_0000	64 KB	LEDs GPIO
0x4180_0000	64 KB	Interrupt controller
...	...	...

Offset	Register	Meaning
0x000	GPIO_DATA	Pin values
0x004	GPIO_TRI	Direction (not present on our blocks)
0x11C	GIER	Global interrupt enable
0x120	IP_ISR	Interrupt status
0x128	IP_IER	Interrupt enable

